Note on Travel Salesman Problem *

Zhongkui Ma

February 19, 2022

1 Introduction

The Traveling Salesman Problem (TSP) states the problem: for a given number of cities and distances between cities, how to choose a route that passes through each city and returns to the starting point, which is required to have the shortest total distance. It is an NP-hard problem in combinatorial optimization and has great significance in computer science and operations research. Many problems are its promotion. In graph theory, the content of TSP is whether there is a Hamiltonian cycle with the smallest length (weight) in a simple undirected graph with weight (a cycle passing through all nodes of the graph is called a Hamiltonian cycle). In computational complexity theory, TSP determines whether there is a cycle in a graph whose length is less than a given length.

2 Related Concepts

In the following discussion, the default simle graph $G = (V(G), E(G))$ is weighted and undirected. $d_T(i)$ represents the connection degree of the vertex i in the tree T; $d_G(i)$ represents the connection degree of the vertex i in the graph G. $s_T(i)$ represents the distance from the vertex i along the tree T to the root vertex r, which is the weight sum of the edges passed. $w(i, j)$ represents the weight of the edge (i, j).

Definition 1 *In the graph G, the union of the shortest path $\{S(r, i)\}$, obtained by the shortest path algorithm to all other vertices $i \in V$, is called the* ***shortest-path tree*** *$T(r) = (V(T), E(T))$; that is $T(r) = \cup_{i \in V} S(r, i)$.*

Definition 2 *For a graph G and a tree $T(r) = (V(T), E(T)) \subseteq G$, the parent vertex of vertex i is vertex k. In the adjacent vertex $\{j|(i, j) \in E(G) - E(T)\}$ of the vertex i, if there is a vertex $j \neq k$ that meets the following conditions (1), then change the parent vertex of i to j, that is, delete the edge $(k, i) \in E(T)$ and add the edge $(i, j) \in E(T)$.*

$$j = \min_{j \in \{j\}} \{s'_T(j) = s_T(j) + w(i, j) | s'_T(j) > s_T(i)\} \tag{1}$$

This operation is called ***minimum increment reconnection****.*

Definition 3 *If the vertex i satisfies: (1) its parent vertex k is not the root vertex, that is, it satisfies $k \neq r$ and $d_T(k) > 2$; (2) it has an adjacent edge $(i, j) \in E(G) - E(T)$, and one of the following two conditions is true: the adjacent vertex j is not a root vertex but a leaf vertex, that is, it satisfies $d_T(j) = 1$ and $j \neq r$; the adjacent vertex j is the root vertex, that is, $j = r$ is satisfied. Reconnect all the adjacent vertices $\{j\}$ that meet the conditions with the smallest increment, and delete the edge (k, i) in the graph G. This operation is called* ***forward connection****.*

*This note does not guarantee the correctness and you can email to me (*zhongkui.ma@uqconnect.edu.au*) if you are interested or have problems with it.*

Definition 4 *If the vertex i satisfies: (1) it is a leaf vertex of the shortest-path tree T, that is, it satisfies $d_T(i) = 1$; (2) it has an adjacent edge $(i, j) \in E(G) - E(T)$, and the adjacent vertex j is not a root vertex but a leaf vertex, that is, satisfies $d_T(j) = 1$ and $j \neq r$; for all the adjacent vertices $\{j\}$ that meet the conditions are reconnected with the smallest increment, and the edges (k, i) are also deleted in the graph G. Furthermore, repeat this step for the original parent vertex k until the condition is not met. This operation is called* ***backward connection****.*

Definition 5 *c is a child vertex of i, if vertex i satisfies: (1) it has an adjacent edge $(i, j) \in E(G) - E(T)$, And the adjacent vertex j is a leaf vertex; (2) it is the ancestor vertex of the adjacent vertex j but not the parent vertex; (3) $w(i, j) > w(i, c) + s_T(j) - s_T(i)$, where $s_T(j) - s_T(i)$ is the weight (length) of the side of the flipped path. Then delete the edge $(c, i) \in E(T)$. And set the parent vertex of vertex j to be i, that is, add edge $(i, j) \in E(T)$. This operation is called* ***flip connection****.*

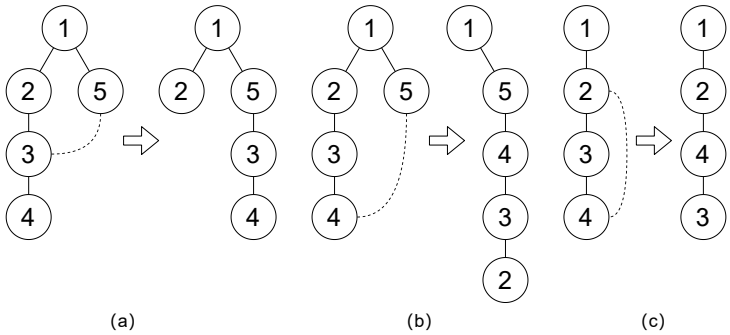

Figure 1: Three Sorts of Connection ((a) is the forward connection; (b) is the backward connection; (c) is the flip connection.)

3 Algorithm

3.1 Description

The overall idea of the algorithm is to continuous ly adjust the edges of the tree according to the shortest-path tree $T(i) = (E(T), V(G))$ and the graph G, Finally, if the tree is a path and there is one edge between the leaf vertex and the root vertex, the cycle R in the graph G can be obtained. The content of the algorithm is described below.

1. First set a root vertex $r \in G$, and obtain the shortest-path tree $T(r)$ with this vertex as the starting vertex.

2. Traverse all the vertices, judge whether the current vertex i is satisfied for backward connection, forward connection, or flip connection, and calculate and record the weight increment Δw generated for the shortest-path tree T after performing these operations. If a vertex allows multiple operations, record the smallest weight increment and its corresponding operation.

3. Select the vertex i with the smallest incremental weight among all vertices, and perform one of the three operations.

4. Repeat steps 2 and 3 until all vertices do not meet the conditions of the three operations.

5. Traverse all the vertices, check whether the tree $T(r)$ is a path, and further check whether the root vertex r and the only leaf vertex i have edges $(r, i) \in E(G)$. If these conditions are met, the graph $T \cup (r, i)$ is the cycle to be found; otherwise, there is no Hamiltonian cycle in the graph G.

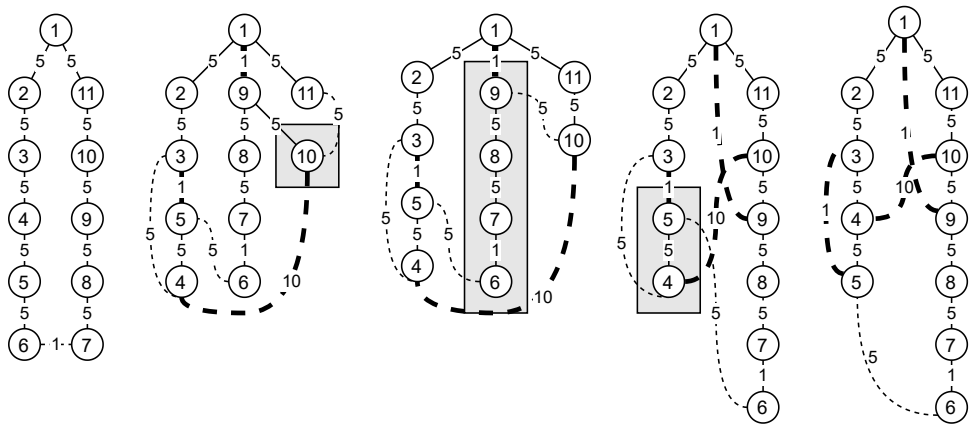

Figure 2: An Example (The first graph is the targeted Hamiltonian circle. The second graph is the initial graph. The thick dotted line refers to the edges not belong to the targeted circle. The second graph shows the backward connection; the third graph shows the forward connection; the third graph shows the flip connection. These graph shows results of different connection and the Hamiltonian circle has been constructed obviously. After another backword connection, the tree will be a path and the targeted circle can be obtained.)

3.2 Proof

The shortest-path tree T of a circle R is a tree with only two branches or one path. If the vertices on the cycle $R = (V(R), E(R))$ are continuously increasing, the edges connect the non-adjacent vertices, then the shortest-path tree $T = (V(T), E(T))$ and the graph $G = (V(G), E(G))$ (it was R at the beginning) will change. This process becomes the **construction process**. When adding some relatively long edges, the shortest-path tree will not change; the shortest-path tree will change when adding some relatively short edges. The reverse process of this process is called the **deconstruction process**. The change of the shortest-path tree is the three changes in the algorithm, forward connection and backward connection and flip connection. The anti-change of these three changes is still itself.

Theorem 1 *The deconstruction process in the algorithm corresponds to the construction process of the Hamiltonian circle with the smallest weight.*

Proof *In each change, an edge e_r has been deleted, and an edge e_a has also been added. This e_r should change $E(T)$ during the construction process, replacing the original e_a. The reason for the change is that e_r provides shorter paths for some vertices without changing the shortest paths of other vertices. The reverse process of this change is that e_a replaces e_r, which increases the length of the shortest path of some vertices without changing the shortest path of other vertices. This increase should be minimal. There are many $\{e_a\}$ that can replace e_r, then choose the vertex that can cause the smallest incremental weight. This process is called minimum increment reconnection. In both the forward and backward connections, the newly connected vertex j may have more than one choice. Here, select the j that produces the smallest increment. In order to obtain the smallest increment, construct the Hamiltonian circle with the smallest weight. The*

new connection vertex j in the flip connection is determined, so there will be no multiple choices, but it still needs to meet the conditions for generating increments, $w(i,j) > w(i,c)+s_T(j)-s_T(i)$.

The entire deconstruction process can be reversed, so a deconstruction process corresponds to a construction process. Although there may be multiple choices in the minimum increment reconnection process, the minimum increment choice ensures that the corresponding construction process must exist. The corresponding construction process is to change the shortest path as little as possible each time. ■

3.3 Complexity

Suppose there are m edges and n vertices in the graph G.The time complexity of the forward connection is $O(1)$, and the time complexity of the backward connection and the flip connection is $O(m+n)$. The time complexity of step 2 is $O(n)$, and it needs to repeat $O(n)$ times. The time complexity of step 3 is $O(n)$. The time complexity of step 4 is $O(n)$. Therefore, the overall time complexity of the algorithm is $O(n^2)$.